COMP2054-ADE

Lecturer: Andrew Parkes

Mergesort:

Algorithm and Cost Analysis

Lec 09

General Algorithm Strategy: Divide and Conquer

- To solve a problem, a strategy that may work is

1. Divide the problem into two (or more) smaller pieces
2. Solve each small piece
3. Conquer the original problem by combining the solutions

And this is done recursively, so the "Solve each small piece" is done by calling the original method, until it is so small it is solved directly.

Divide and Conquer: Mergesort

- Mergesort uses the "Divide and Conquer" strategy
- To sort an array A:

1. Divide the array into two smaller arrays A1 A2
2. Sort each smaller array A1 and A2
3. Conquer the original problem by combining – merging - the sorted smaller arrays, A1 and A2 into one sorted array

And this is done recursively, so the "Sort each smaller array" is done by calling the original method, until it is so small it can be sorted directly.

Mergesort : Pseudo-code

A sketch of the code structure is:

```
mergeSort(int[] A) {
      if ( A.size <= 1 ) return;
      int [] A1, A2
      (A1,A2) = split(A)
      mergesort(A1)
      mergesort(A2)
      A = merge(A1,A2)
}
```

Note that it is “binary recursion” – there are two calls to the mergeSort in the body of the mergeSort

Mergesort : Merge step – basic idea

It is easy to take two sorted arrays and merge them into a single sorted array:

```
merge(int[] A1, int[] A2) {
      int[] A
      while A1 or A2 are non-empty
         compare the "first" elements of A1 and A2
         move it to A
      return A
}
```

But for arrays we want to "ignore in future" rather than actually remove, so instead use moving indices:

Mergesort : Merge – basic idea (2)

It is easy to take two sorted arrays and merge them into a single sorted array:

```
merge(int[] A1, int[] A2) {
      int[] A
      int k1=1, k2 = 0;
         compare the "first" elements of A1 and A2
         move he smallest to A
      return A
}
```

Mergesort: Split using sub-arrays

For the "split" and "passing around of arrays" it is not very efficient to copy all the time. Instead, when we had "Array" we refer to a sub-array of the original array defined by start and end indices:

Example: suppose A.size=8 with indices 0,…,7

Then we take

A1 = subarray with indices 0,1,2,3

A2 = subarray with indices 4,5,6,7

Mergesort: Split using sub-arrays (2)

Example: suppose A.size=8 with indices 0,…,7

A1 = subarray with indices 0,1,2,3

A2 = subarray with indices 4,5,6,7

A1				**A2**			
2	4	5	8	1	5	7	9
^				^			

and will copy into a separate workspace:

The workspace might be passed around via an extra argument to the calls – to save unneeded new/delete costs

Mergesort: Merge using sub-arrays

Work by comparing the values at the pointers (indices) and copying smallest to the workspace and moving that pointer:

A1				**A2**			
2	4	5	8	1	5	7	9
^				^			

Compare 2 and 1, so move 1 and the pointer

1							

A1				**A2**			
2	4	5	8	1	5	7	9
^					^		

Mergesort: Merge using sub-arrays (2)

Work by comparing the values at the pointers (indices) and copying smallest to the workspace and moving that pointer one step to the right:

A1				**A2**			
2	4	5	8	1	5	7	9
^					^		

Compare 5 and 2, so move 2 and also the pointer

1	**2**						

A1				**A2**			
2	4	5	8	1	5	7	9
	^				^		

Mergesort: Merge using sub-arrays (3)

Repeat until both pointers reach the end. If there are ties, then pick the earliest one (WHY!?). If one pointer reaches end first, then just copy the rest of the other:

A1				**A2**			
2	4	5	8	1	5	7	9
^				^			

Repeat until done both sub-arrays

1	**2**	**4**	**5**	**5**	**7**	**8**	**9**

A1				**A2**			
2	4	5	8	1	5	7	9

Mergesort: Merge using sub-arrays (4)

Copy the workspace back to the array – ignoring the division in the middle

Workspace:

1	**2**	**4**	**5**	**5**	**7**	**8**	**9**

Main array becomes:

1	2	4	5	5	7	8	9

And we can now just ignore the marker that was in the middle – so we are finished – i.e.just return the array

Mergesort : Merge –running cost

After each comparison an element will be moved.
This is O(1) per element.
Hence O(n) overall for the comparisons and copy to the workspace

Copy the workspace back to the array is also O(n)

Hence:

When merging two sub-arrays of joint total size n the cost is O(n)

Mergesort : Overall code

Because we are working with sub-arrays it is convenient to implement a function to sort a sub-array. Define the sub-array by the indices of the Left and Right ends:

```
mergeSort(int[] A, int L, int R)
        if ( L=R )  // only one entry, already sorted
           return
      int M = (L+R)/2    // mid point
      mergeSort(A,L,M)  // sort left sub-array
      mergeSort(A,M+1,R) // sort right sub-array
      merge(A,L,M,M+1,R) // nothing else to do!
}
```

Initial call is just to all the array:

```
    mergeSort(int[] A) {mergeSort(A,0,A.length-1);}
```

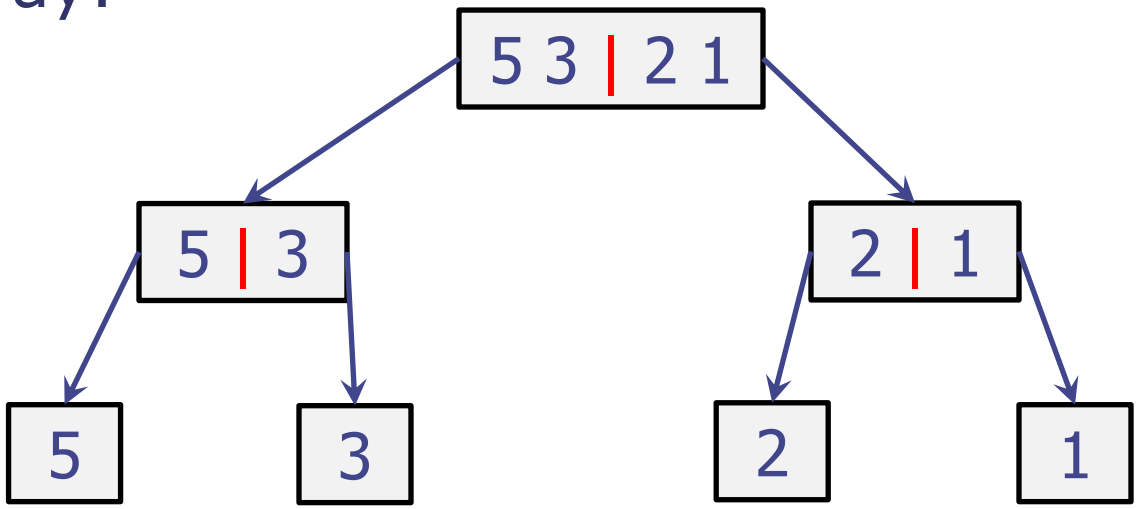
Depiction “trace” of Mergesort run

- Since mergesort is a binary recursion the usual “stack trace” will become a binary tree
- Represents the recursive calls and the effects

- IMPORTANT: The trees in the analysis are purely for depiction of the running of the algorithm.
- In the implementations of mergesort there are no “physical” trees – it is just arrays (or lists)

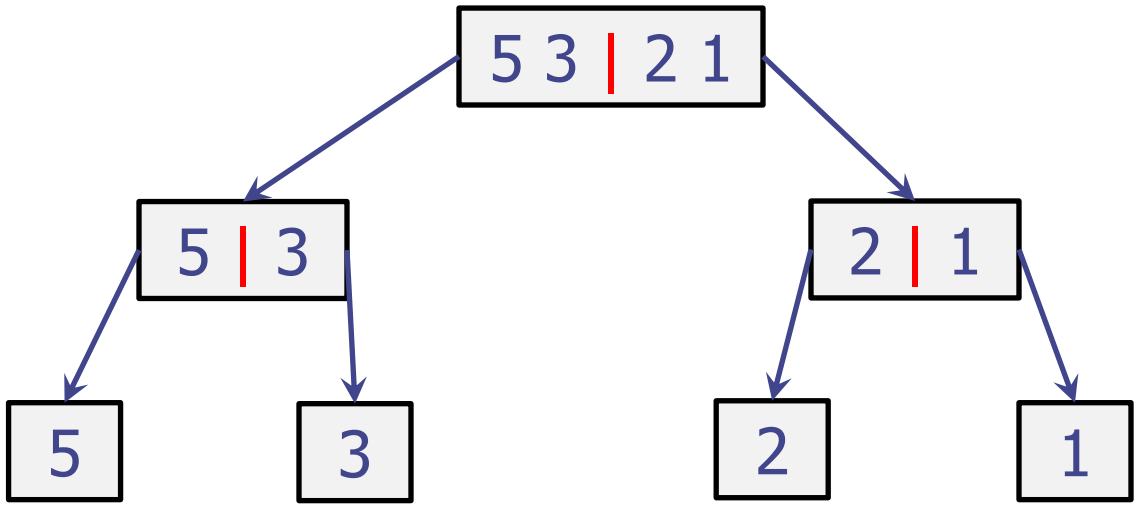
Depiction “trace” of Mergesort run (2)

- Firstly, just show the recursive calls using the partition of the arrays into two – represented here by a “|”
- Partition stops when we reach 1 element per array:



Depiction tree: height?

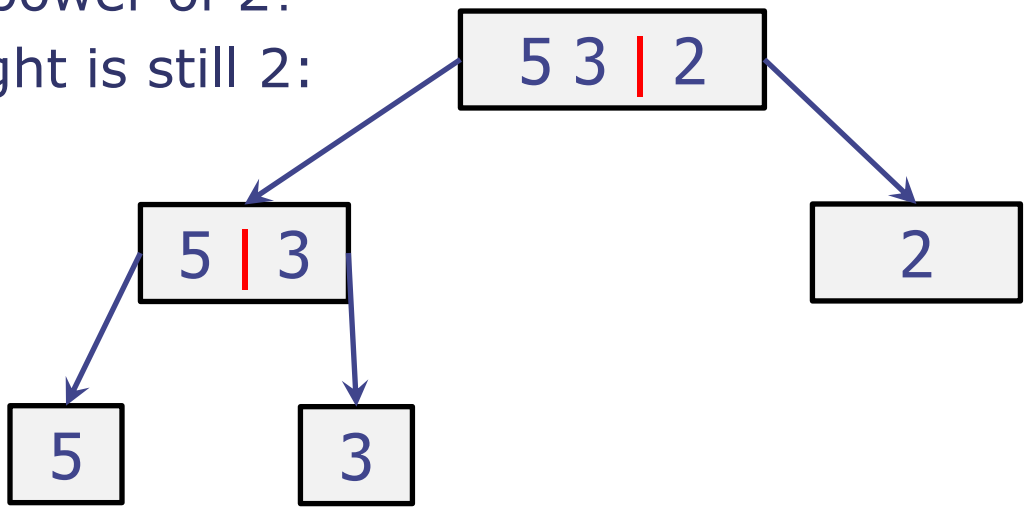
- What is the height of the "Trace Tree"?
- Suppose n = 2^k, then how many times do we need to "divide by two" to reach 1?
- Ans: k. E.g. n=4=2^2 has height 2



- So: height = log_2 (n) when n=2^k

Depiction tree: height for general n?

- Suppose n is not a power of 2?
- E.g. at n=3 the height is still 2:



- So, general n behaves like roundup to the next power of two, and so

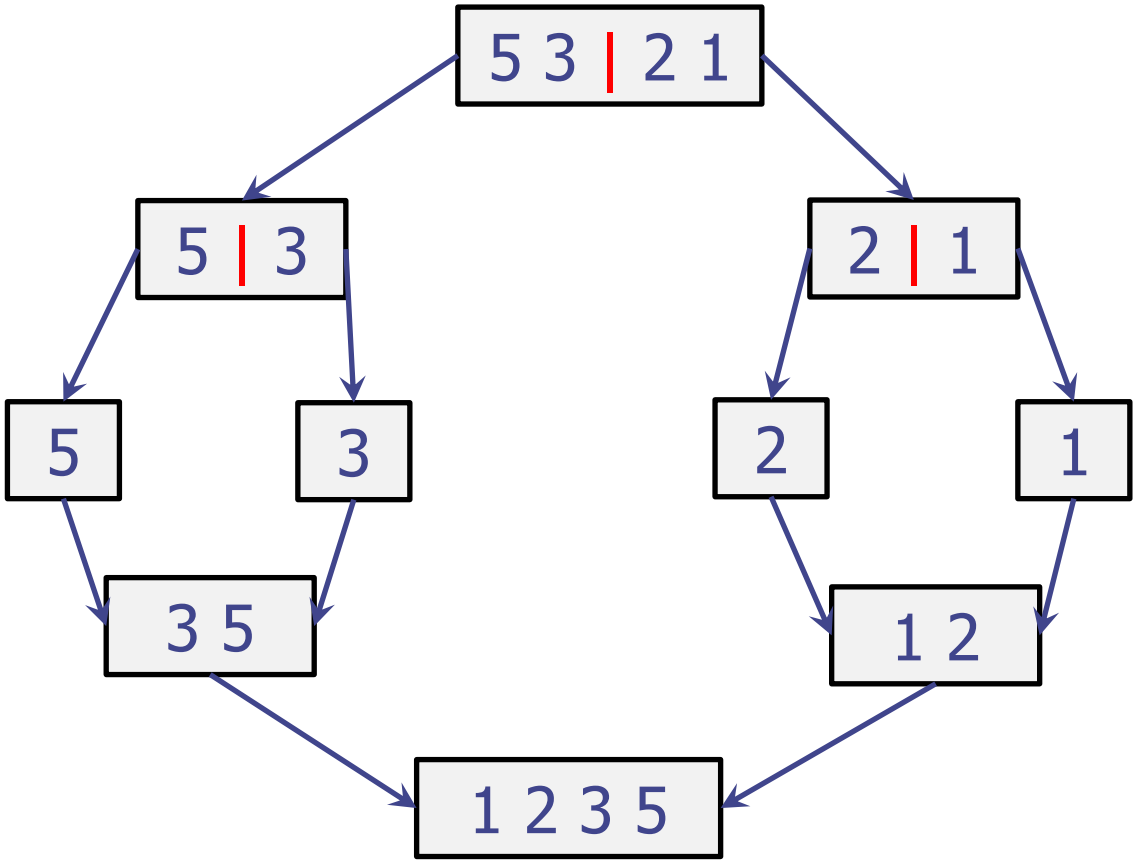
  height = ceiling(log_2 (n))

- "Ceiling" only affects by adding a constant, which would be dominated by the log in Big-Oh analysis – so easier to just do the analysis using = 2^k
- Also, can ignore the base of the log, so can just say:

  height is O(log n) (or Theta(log(n)) to be strict)

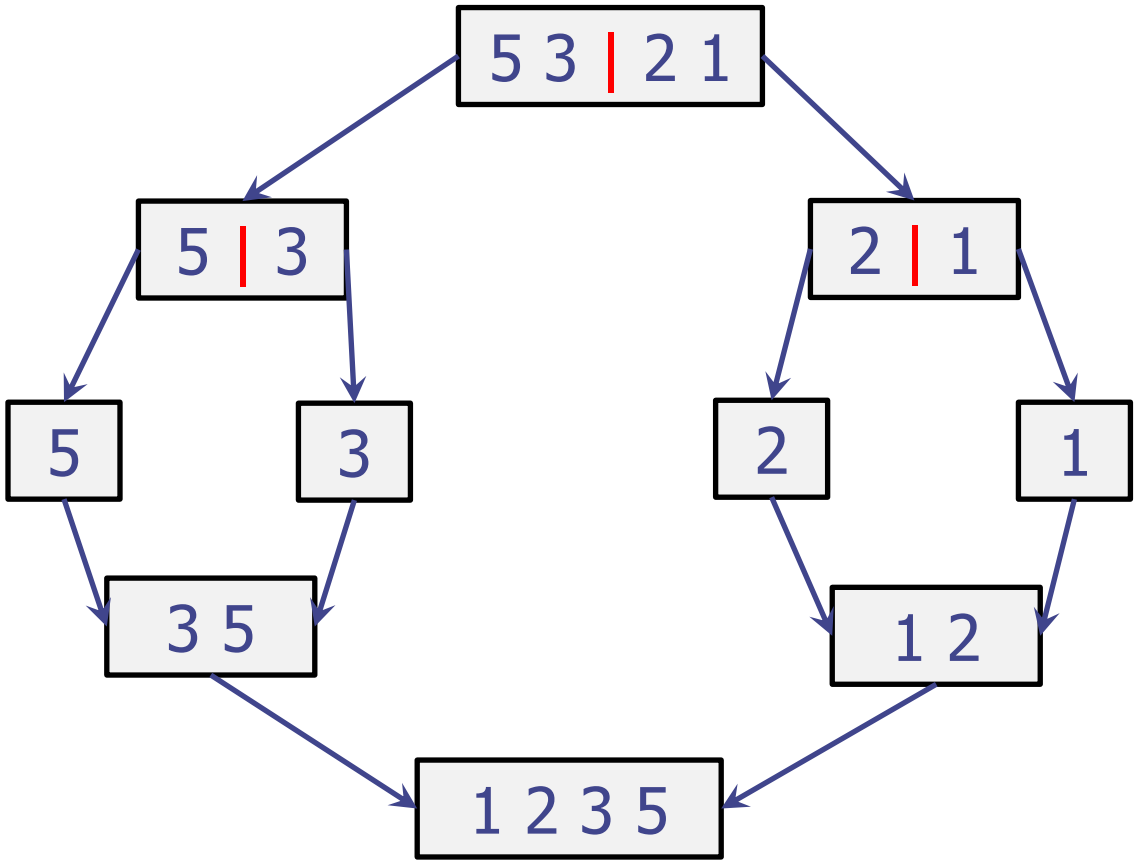
Depiction tree: including the merge

- Need to also do the merge:
  - There will be a second tree that does the merge steps
  - Can show this as an "upside down tree":



Work Analysis: "Divide" steps

- During the division we are putting a "marker" to partition the array
  - This is cheap in an array: will be O(1)
  - Need to place n-1 markers, and so overall takes O(n)



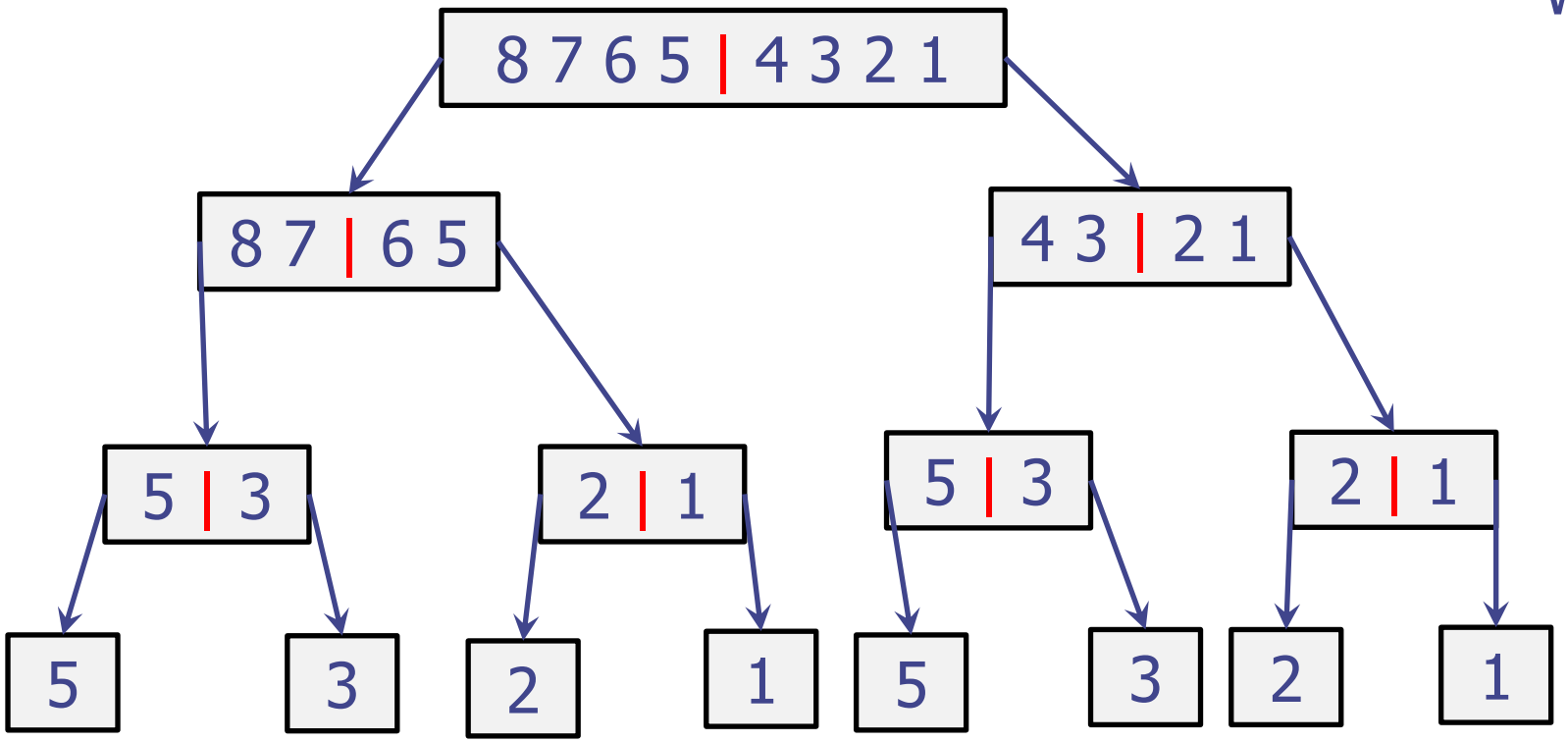
Work
1

2

Total =3
= n-1

Work Analysis: "Divide" steps (2)

- With larger array: n = 2^3 = 8

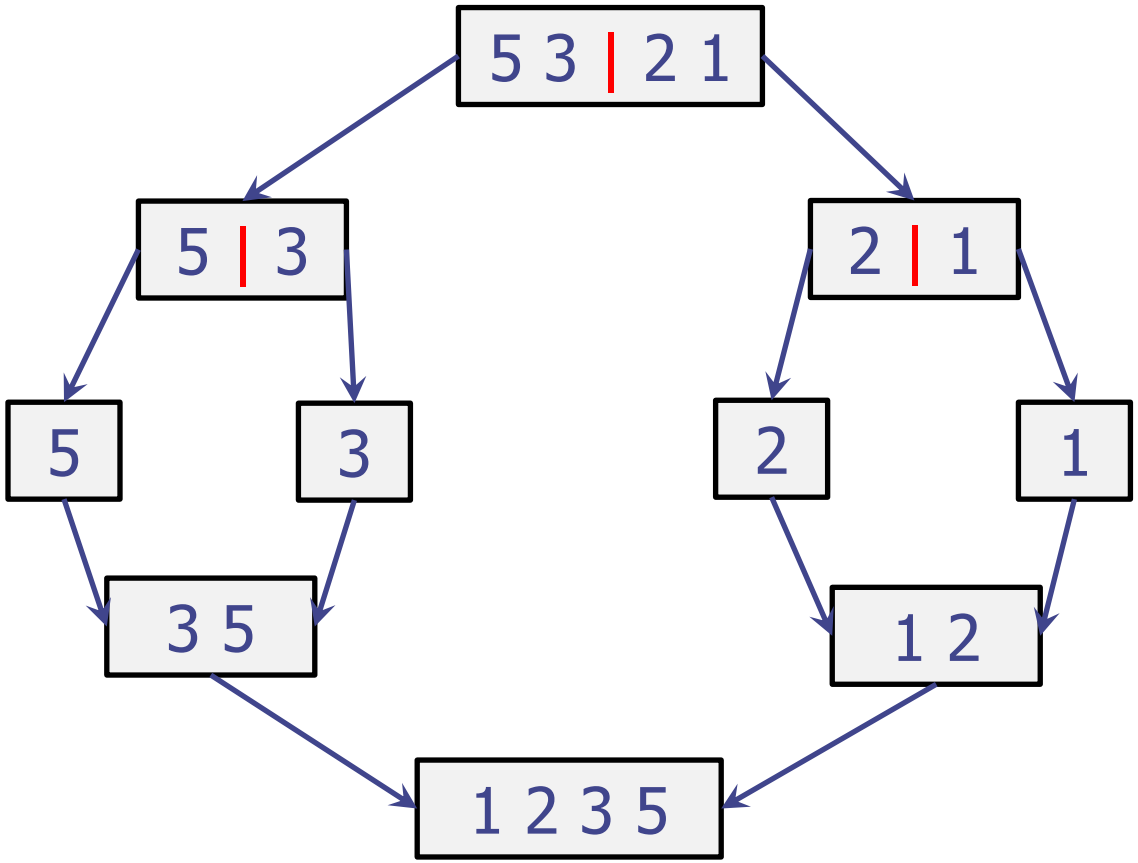


Work per level:
1
2
4

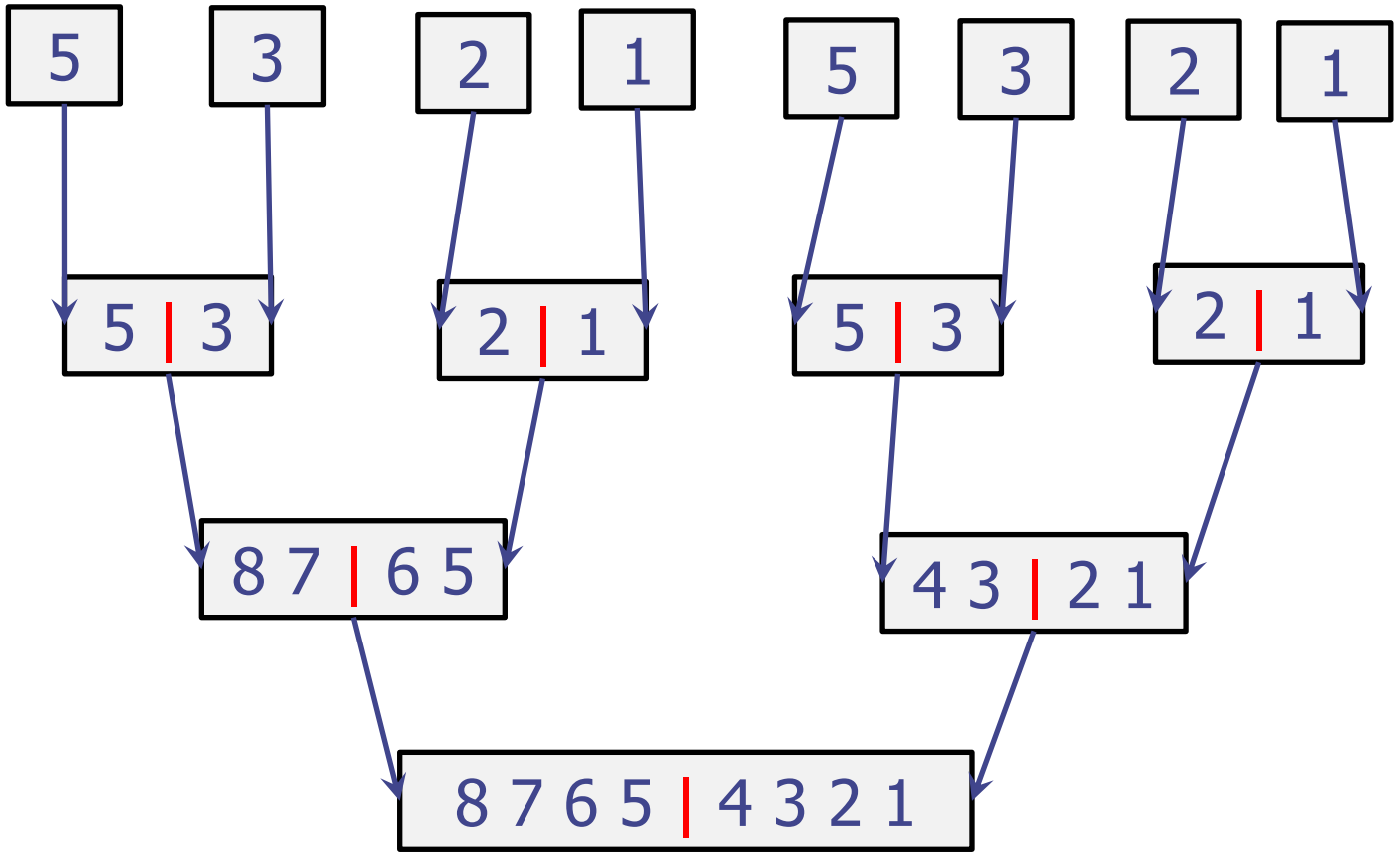
- Height = 3
- Num. of markers = 1+2+4 = 2^0 + 2^1 + 2^2 = 2^3 – 1 = 7

Work Analysis: "Merge" steps

- During the merge we have to process every element of every sub-array
- Let's count elements as they are merged

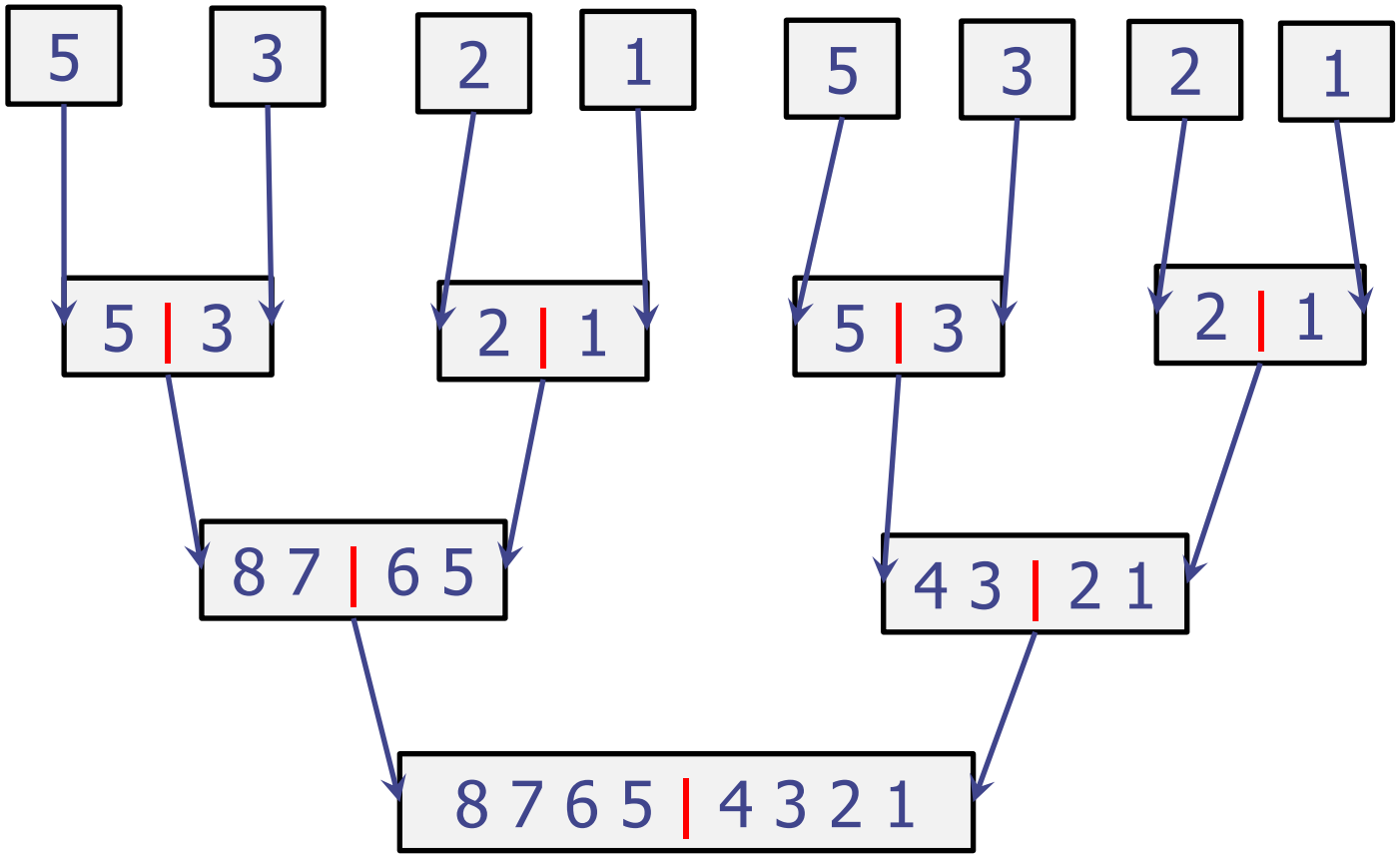


Work Analysis: "Merge" steps (2)



- With larger array: n = 2^3 = 8
- Still get that work per level is O(n)

Work Analysis: "Merge" steps (3)



Work per level:

8

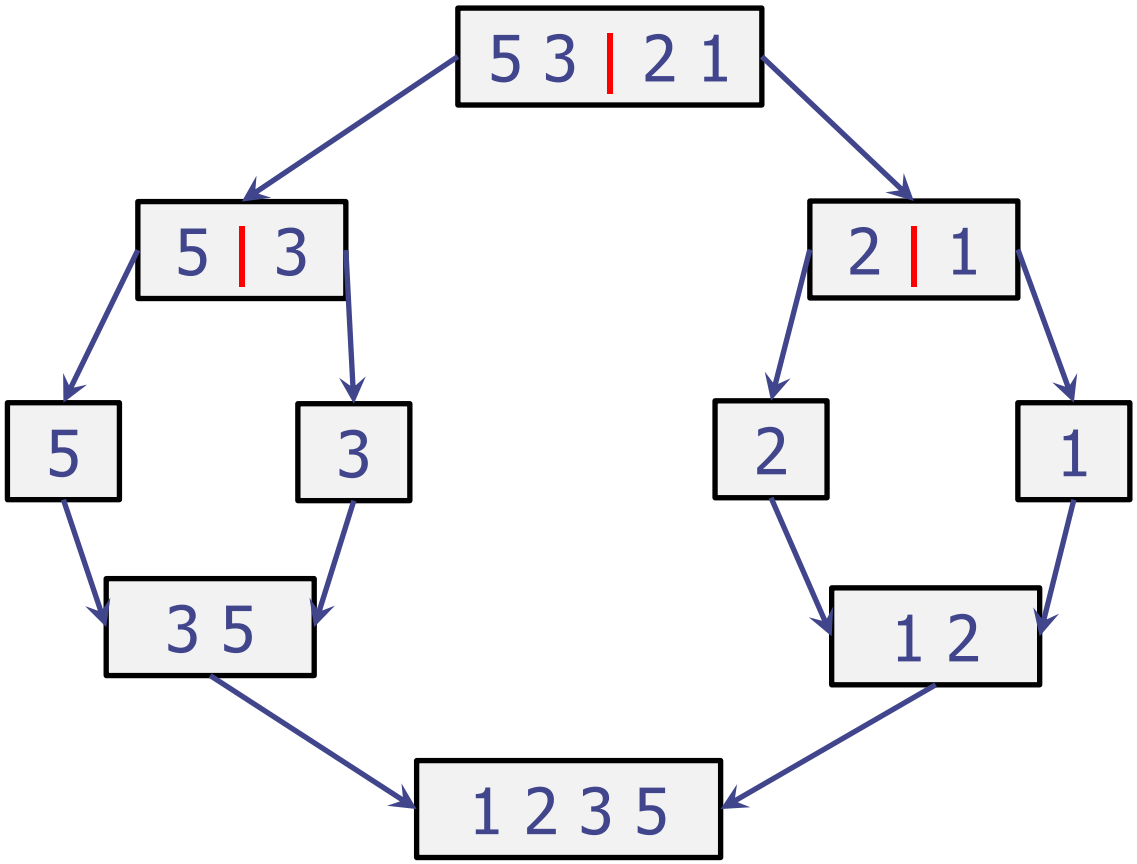
8

8

- Still get that work per level is O(n)
- Number of levels is O(log n)
- Total work is O(n log n)

Depiction tree: Total Work

- All each divide step we just do O(1) work, but each merge is O(n) per level



Work per level:

1

2

4

4

Depiction tree: Total Work (2)

- All each divide step we just do O(1) work
- But each merge is O(n) per level

- Total work per level is O(n)

- There are O(log n) levels
  - As height of a (nearly full) binary tree

- Overall, **mergesort is O(n log n)**

Using merge sort

- Fast sorting method for arrays
- Good for sorting data in external memory – because works with adjacent indices in the array (data access is sequential)
  - It accesses data in a sequential manner (suitable for sorting data on a disk)
- Not so good with lists, because it relies on constant time access to the middle of the sequence

- What else can we say about mergesort?

Mergesort is stable

- Recall: “stability” means the ordering of ‘equal’ items (under the comparison function) is not changed by the sorting

- In merge-sort we only move items during the merge phase
- So, consider an example of the merge:

Mergesort : Stability

A1				**A2**			
2	4	5	8	2′	6	7	9

Merge Implementation: If items are equal, then take the leftmost one.

Gives

2							

then (check this!) the next step takes 2′

2	**2′**						

So, 2 is still to the left of 2′.

Hence, this implementation is stable

"Adaptive" merge sort ?

- Recall "Adaptive" means "can exploit when a list is 'nearly sorted' "
  - i.e. adapts to the data being "partially sorted"
  - E.g. when a large fraction of the adjacent pairs are already ordered
- What is the best case of Mergesort?
  - For the "naïve" mergesort (as given in these slides), the best case is no better than the worst case
    - it still goes through the entire process even if the original list is already sorted ☹
    - Hence, basic mergesort is not adaptive
    - Exercise (offline): think of one (easy) change to make it more adaptive

Mergesort: Runs and Merging

- Think about the properties of the array when we are still in the middle of doing the mergesort
- It will have subarrays that are already ordered
  - **Call an ordered subarray a "run"**
- In standard mergesort the initial "divide" breaks the array into subarrays of size 1
  - These are automatically "runs"
  - Mergesort then **merges together adjacent runs**
- Example:

Mergesort : “half way through”

Halfway through mergesort, the first half of the array will be sorted; i.e. have a long run with many small ones. E.g.

A1				**A2**	**A3**	**A4**	**A5**
2	4	5	8	3	6	9	7

Mergesort will then continue with:

1. merge(A2,A3) - gives a run, call it A6
2. merge(A4,A5) - gives a run, call it A7
3. merge(A6,A7) - gives a run, call it A8
4. Merge(A1, A8) - gives a run of the total size – so are finished

Observe: Mergesort split subarray [3,6] into A2 and A3, and then merged them again even though it was already sorted. This was inefficient!

“Timsort”: Basis of the sort in Python

Basic idea of “Timsort” is based on mergesort:
Interleave two kinds of steps:

1. Find runs, add them to some data structure (a stack)
   - Walks the array until find an out-of-order pair
   - Such walks do not do any sorting and so are just O(|run|)
2. Merge together adjacent runs
   - Follow some policy for which ones to merge
   - Until recently (2015!) the policy used in Python had a bug!
     - Fixed version is called “Powersort”

If array is already totally sorted, it will just find one run and stop - taking just O(n) steps – beats the usual O(n log n)

When an array is nearly sorted, then:

- Many runs will be long
- Not so many merges will be needed
- Overall, the sorting will be faster

Motivation for COMP2065-IFR

https://en.wikipedia.org/wiki/Timsort#Formal_verification

> *"Using the KeY tool for **formal verification** of Java software, the researchers found that this check is not sufficient, and they were able to find run lengths (and inputs which generated those run lengths) which would result in the invariants being violated deeper in the stack after the top of the stack was merged."*

https://ercim-news.ercim.eu/en102/r-i/fixing-the-sorting-algorithm-for-android-java-and-python

> *"by Stijn de Gouw and Frank de Boer*
> *In 2013, **whilst trying to prove the correctness of TimSort** - a broadly applied sorting algorithm - the CWI Formal Methods Group, in collaboration with SDL, Leiden University and TU Darmstadt, **instead identified an error in it, which could crash programs and threaten security**. Our bug report with an improved version, developed in February 2015, has led to corrected versions in major programming languages and frameworks"*

Questions to ask about sorting algorithms

- Big-Oh complexity (both time and space)?
  - Best case inputs? Worst case inputs?
- Extra workspace needed? Or is it `in-place'? (To be explained in "quicksort" lecture)
- Stable sort?
- "Adaptive sorting" – how well does it do if the data is already "nearly sorted"
- Data access patterns?
  - Sequential? Random Access?
- (Not assessed: Parallel versions?)
- (Not assessed: Relevant and appropriate "assertions" – what is true at various stages. Assertions can help with understanding and with checking correctness.)

Should aim to understand these issues for various sorting algorithms.

Minimum Expectations

- Know the Mergesort algorithm and how it works on examples
- Know and be able to justify/prove the big-Oh family of behaviours of Mergesort
- Know about the stability / adaptive properties